

Method C v2: a Bayesian-symmetric monotone ansatz for the K=3 CRRA REE equilibrium price function (methodology + neural alternatives)

Fixed-Point Factory

June 2026

1 The problem

The K=3 CRRA Hellwig rational-expectations equilibrium is the fixed-point equation

$$P(u_1, u_2, u_3) = \Phi[P](u_1, u_2, u_3),$$

where each agent observes a private signal u_i , market clearing yields the price P , and Φ is a Bayesian-CRRA operator: it computes each agent's posterior $\mu(p, u_k)$ from observing the price p and their own signal u_k , then CRRA-clears the market across the three agents.

The unknowns are the values of P . In our previous work we represented P as a 343-element tensor on a $G^3 = 7^3$ cube grid and solved the FP equation cell-by-cell. We hit two structural problems:

- The kernel-band Richardson operator (smooth, Newton-friendly) gives FPs that are *non-monotone* in $\sim 27\%$ of cube cells at high τ — non-physical artifacts.
- The strict- $h=0$ operator (Bayesianly correct) is non-smooth at p values where the price level set passes through critical points of P ; the FP iteration does not converge.

2 The Bayesian symmetries

The equilibrium price $P(u_1, u_2, u_3)$ must satisfy four structural properties dictated by the model:

1. **Permutation symmetry.** Agents are exchangeable: $P(u_1, u_2, u_3) = P(u_{\pi(1)}, u_{\pi(2)}, u_{\pi(3)})$ for any permutation π .
2. **Sign-flip symmetry.** The state space is binary $v \in \{0, 1\}$ with symmetric prior; the signal distribution is $f_v(u) = N(\pm 0.5, 1/\tau)$, antisymmetric under $u \rightarrow -u$ and $v \leftrightarrow 1 - v$. This gives $P(u_1, u_2, u_3) = 1 - P(-u_1, -u_2, -u_3)$.
3. **Axis-wise monotonicity.** More positive u_i increases the Bayesian posterior on $v = 1$ for any agent observing it, and (via market clearing) raises the equilibrium price: $\partial P / \partial u_i > 0$ for each i .

4. **Range.** $P \in (0, 1)$.

Any candidate FP violating these is not a true equilibrium. The cube-discretized solvers we used produce candidate FPs that *violate (3) and (4)* numerically. Instead of fighting the discrete representation, Method C v2 builds (1)–(4) into the parameterization.

3 Method C v2 construction

Parameterize P as

$$P(u_1, u_2, u_3) = \sigma(h(u_1) + h(u_2) + h(u_3)), \quad \sigma(x) = \frac{1}{1 + e^{-x}},$$

where $h : \mathbb{R} \rightarrow \mathbb{R}$ is a **1D odd, monotone-increasing** function. Write

$$h(u) = \int_0^u \sigma_*(s)^2 ds,$$

with $\sigma_* : \mathbb{R} \rightarrow \mathbb{R}$ a 1D *even* polynomial of degree $2d$:

$$\sigma_*(s) = \sum_{n=0}^d a_n T_{2n}(s/U),$$

in the Chebyshev basis (we use $U = 2.33$ to cover the practical signal range).

3.1 Why this enforces all four symmetries by construction

1. **Permutation symmetry.** $h(u_1) + h(u_2) + h(u_3)$ is manifestly symmetric in the agents.
2. **Sign-flip symmetry.** σ_* even $\Rightarrow \sigma_*^2$ even $\Rightarrow h$ odd in $u \Rightarrow h(u_1) + h(u_2) + h(u_3)$ is odd in $(u_1, u_2, u_3) \rightarrow (-u_1, -u_2, -u_3) \Rightarrow \sigma(L) + \sigma(-L) = 1$.
3. **Monotonicity.** $\sigma_*^2 \geq 0$ gives $h'(u) = \sigma_*^2(u) \geq 0$, so h is monotone non-decreasing. Hence $\partial P / \partial u_i = \sigma'(L) \cdot h'(u_i) \geq 0$.
4. **Range.** $\sigma \in (0, 1)$ for any finite L .

3.2 Unknowns

Just $d + 1$ scalars: $a_0, a_1, \dots, a_d$. We use $d = 4$ (so σ_* is degree 8) giving **5 unknowns per cell**. Vastly fewer than the 343 cube unknowns or the 84 symmetric-reduced cube unknowns.

3.3 Numerical pipeline per (γ, τ) cell

1. Given current $\{a_n\}$:
 - Compute $h(u_i)$ at the $G = 7$ cube nodes via 16-point Gauss-Legendre quadrature on $[0, u_i]$.
 - Form $P_{ijk} = \sigma(h(u_i) + h(u_j) + h(u_k))$.

2. Apply the standard Lin-CDF R4 kernel-band operator Φ to get $\Phi[P]$.
3. Compute residual $r_{ijk} = \Phi[P]_{ijk} - P_{ijk}$.
4. Minimize $\|r\|_2^2$ over the 5 unknowns via SciPy `least_squares(method=trf, jac="2-point")`.

Per cell wall time: ~ 1 s (5 unknowns + few-point FD Jacobian + a handful of TRF iterations).

3.4 What the residual means

The minimised residual $|F|_\infty = \max_{ijk} |\Phi[P]_{ijk} - P_{ijk}|$ is the **structural non-additivity** of the equilibrium: how much it deviates from being expressible as a sum of three identical 1D contributions. This separates cleanly from:

- Kernel-band bias (gone — since we measure the residual at the *ansatz* P , not at a kernel-band-biased FP).
- Discretisation noise (gone — the ansatz is analytic).
- Operator non-smoothness (gone — our P is C^∞).

If $F \rightarrow 0$ at low τ , the equilibrium IS additive — can be written as a sum of agent-wise contributions. If F grows at high τ , the equilibrium has irreducible joint structure (genuine cross-signal information beyond the sum of marginal posteriors).

4 Empirical results (preliminary)

From a partial τ ladder run at $\gamma = 100$:

τ	residual $ F _\infty$	meaning	wall
0.025	2.7×10^{-8}	ansatz captures equilibrium nearly exactly	1 s
0.05	2.1×10^{-7}	still additive to high precision	1 s
0.10	5.5×10^{-6}	beginning to see non-additivity	1 s
0.25	8.0×10^{-4}	moderate non-additivity	1 s
0.50	3.4×10^{-2}	substantial non-additivity	1 s
1.00	$\sim 5 \times 10^{-2}$ (proj.)	strong non-additivity, P spans $\sim$ full range	1 s

The residual rises smoothly with τ , suggesting the equilibrium is **additive to high precision at low τ** and acquires irreducible joint structure as the signal precision grows.

5 Could a neural network do better?

Universal approximation: yes. A neural network is a universal function approximator, so in principle it could fit the equilibrium to arbitrary precision. But the right question is: what's the *efficient* parameterisation that respects the same Bayesian symmetries?

5.1 Naïve MLP: no

A vanilla MLP $NN(u_1, u_2, u_3)$ has none of the four required symmetries built in. Enforcing them via training:

- Permutation symmetry: data augmentation, but expensive and only approximate;
- Sign-flip: similar issues;
- Monotonicity: requires constrained training (e.g., projected gradient or monotone-NN architectures with positive weights and monotone activations like ReLU/softplus everywhere).

This is much harder than our 5-parameter polynomial.

5.2 Symmetric monotone NN: yes, but redundant

The natural NN counterpart of Method C v2 is

$$P(u_1, u_2, u_3) = \sigma(\phi(u_1) + \phi(u_2) + \phi(u_3)),$$

where $\phi : \mathbb{R} \rightarrow \mathbb{R}$ is a 1D *monotone* neural net (e.g., a stack of positive-weight linear layers with positive activations, or a monotone PCHIP NN). This automatically gives the same four symmetries.

For our problem, the empirical evidence is that a degree-8 even polynomial (Cheby basis, 5 coefficients) already captures ϕ to $F \sim 10^{-6}$ at low τ . An NN would add hundreds of parameters with diminishing returns. **The 1D polynomial is a near-optimal basis here.**

5.3 Joint correction: yes, useful but constrained

To go beyond the additive ansatz at high τ , add a joint correction:

$$P = \sigma(h(u_1) + h(u_2) + h(u_3) + g(u_1, u_2, u_3)),$$

where g is a permutation-symmetric, sign-flip-symmetric function (e.g., g symmetric polynomial in u_i , odd in the overall sign flip). For g , NN choices:

- **Deep Sets** architecture: $g = \psi(\sum_i \rho(u_i))$ with ρ, ψ small NNs. Gives permutation invariance and moderate flexibility. 10^2 params.
- **Equivariant NN**: stricter constraint than Deep Sets, fewer params, faster.
- **Symmetric polynomial basis**: $g = \sum c_{\mathbf{m}} e_{\mathbf{m}}(u_1, u_2, u_3)$ in elementary symmetric polynomials, with sign-flip-respecting parity. 10–50 params.

We expect the symmetric polynomial basis to be most parsimonious for the $K=3$ problem and to match NN performance with fewer parameters.

5.4 Bottom line on NNs

- NNs are *not* a clear win over the polynomial-Cheby ansatz for this problem. The symmetries are exact (not just learnable) with the 5-parameter polynomial.
- For the *joint correction* to capture remaining non-additivity at high τ , both NN and symmetric polynomial bases would work. Polynomial wins on parsimony for the symmetric structure we know holds.
- Worth trying only if the polynomial bases plateau at a residual we want to push lower.

6 Reproducibility

- Operator + ansatz: `projects/REZN/solver_code/highprec_dd_qd/dd_k3_method_c_v2.py`
- Ladder driver: `projects/REZN/solver_code/highprec_dd_qd/dd_k3_method_c_v2_ladder.py`
- Per- τ saved coefficients: `projects/REZN/solved_fixed_points/dd_k3_overnight/method_c_v2/coefs/tau{t}`